

Experiment 4: Harris Corner Detection in Grayscale Images

Aim:

To manually compute and visualize Harris corner responses in a grayscale image.

Theory:

The Harris Corner Detection algorithm is used to identify points in an image where there is a significant change in intensity in all directions — typically corners or junctions. It works by computing image gradients (I_x , I_y), their products (I_{xx} , I_{yy} , I_{xy}), and then applying a mathematical formula to calculate a response value (R) for each pixel. High values of R indicate strong corners. These are then filtered using a threshold and visualized.

Algorithm:

- Start
- Import required libraries (`cv2`, `numpy`)
- Read input image and convert it to grayscale
- Compute image gradients using Sobel operator (I_x , I_y)
- Calculate I_{xx} , I_{yy} , and I_{xy} (squared gradients and product)
- Apply Gaussian blur to these matrices
- Compute Harris response using the formula: $R = \det(M) - k(\text{trace}(M))^2$
- Apply threshold to extract strong corner points
- Mark corners on the image (in grayscale, RGB, and BGR versions)
- Display the results
- End

Flowchart:

Start → Import Image → Convert to Grayscale → Compute I_x and I_y → Calculate I_{xx} , I_{yy} , I_{xy} → Apply Gaussian Blur → Compute Harris R → Threshold → Mark Corners → Display Output → End

Viva Questions:

- What is the mathematical formula for Harris response?
- Why are Sobel derivatives used before Harris detection?
- How is thresholding used to identify corners?